

Tuple and Dictionaries in Python



Algoritma dan Pemrograman II

Team Teaching:
Lia Farhatuaini, M.Kom
Sokid, M.Kom

TIPE SEQUENCE (SEQUENCE TYPE)

- Tipe sequence merupakan suatu tipe data pada python yang memungkinkan untuk menyimpan lebih dari satu nilai (atau kurang dari satu nilai, karena urutan/sequence dapat kosong), dan nilai-nilai ini dapat ditelusuri secara berurutan elemen demi elemen.
- Kita tahu bahwa perulangan for merupakan tools yang digunakan untuk mengiterasi urutan sehingga, suatu sequence/urutan adalah data yang dapat dipindai melalui perulangan for.
- List merupakan contoh klasik dari suatu **python sequence**. dan masih ada sequence lain yang perlu kita pelajari.

MUTABILITAS (MUTABILITY)-1

- Mutabilitas adalah sifat dari data Python yang menggambarkan kesiapannya untuk diubah secara bebas selama eksekusi program.
- Ada 2 jenis data python: **mutable** dan **immutable**.
- **Data mutabel** merupakan data yang dapat diperbarui secara bebas kapan saja (*in situ*)
- *in situ* adalah frasa Latin yang secara harfiah berarti *di tempat*. berikut contoh merubah data secara *in situ*.

```
list.append(1)
```

MUTABILITAS (MUTABILITY)-2

- Data immutable tidak dapat dimodifikasi dengan cara seperti di slide sebelumnya.
- Bayangkan jika sebuah List hanya dapat diberi nilai dan dibaca saja, kita tidak akan bisa menambah atau menghapus elemen List tsb. Jika ingin menambahkan elemen baru di akhir List maka memerlukan pembuatan ulang List.
- Jenis data seperti itu disebut **Tuple**. Tuple merupakan jenis sequence yang **immutable**. Tuple dapat berperilaku seperti List, tetapi tidak bisa dimodifikasi secara *in situ*.

TUPLES

- Perbedaan pertama dan paling jelas antara List dan Tuple adalah sintaksis yang digunakan untuk membuatnya.
- Tuple menggunakan tanda kurung, sedangkan List menggunakan tanda kurung siku.
- Memungkinkan juga membuat tuple hanya dari sekumpulan nilai yang dipisahkan oleh koma.
- Cobalah contoh di bawah dan tampilkan hasilnya.

```
tuple_1 = (1, 2, 4, 8)
tuple_2 = 1., .5, .25, .125
```

Note: setiap elemen pada tuple dapat memiliki tipe data yang berbeda.

HOW TO CREATE A TUPLE

Kita dapat membuat tuple kosong. Dalam hal ini tanda kurung diperlukan.

```
1 tuple_kosong = ()
```

Jika kita ingin membuat tuple dengan satu elemen, maka harus diakhiri dengan koma supaya tuple harus dapat dibedakan dengan variabel biasa yang memiliki nilai tunggal. menghilangkan koma akan berpengaruh kepada program karena bukan membuat tuple melainkan variabel biasa.

```
1 tuple_1 = (1, ) #tuple dgn elemen tunggal
2 tuple_2 = 1, #tuple dgn elemen tunggal
3 tuple_3 = 1 # ini bukan tuple, tp variabel biasa dengan nama tuple_3
```


HOW TO USE A TUPLE-1

- Jika kita ingin membaca elemen dari sebuah tuple, maka dapat menggunakan cara yang sama saat menggunakan List.
- Jangan mencoba untuk memodifikasi isi dari tuple

```
3 my_tuple.append(10000)
4 del my_tuple[0]
5 my_tuple[1] = -10
```

```
1 my_tuple = (1, 10, 100, 1000)
2
3 print(my_tuple[0])
4 print(my_tuple[-1])
5 print(my_tuple[1:])
6 print(my_tuple[:-2])
7
8 for elemen in my_tuple:
9     | print(elemen)
```

HOW TO USE A TUPLE-2

- apa lagi yang bisa dilakukan oleh tuple?
 - fungsi `len()` :
mengembalikan jumlah elemen dari tuple
 - operator `+` : join
(menggabungkan) tuple
 - operator `*` : mengalikan
tuple (sama seperti List)
 - operator **`in`** dan **`not in`**:
bekerja sama seperti pada List

```
1  my_tuple = (1, 10, 100, 1000)
2
3  t1 = my_tuple + (10000, 100000)
4  t2 = my_tuple * 3
5
6  print(len(t2))
7  print(t1)
8  print(t2)
9  print(10 in my_tuple)
10 print(-10 not in my_tuple)
```


HOW TO USE A TUPLE-3

- Salah satu sifat tuple yang paling berguna adalah kemampuannya untuk muncul di sisi kiri operator penugasan (penugasan simultan). salah satu aplikasi paling umum dari penugasan simultan menggunakan tuple adalah untuk menukar nilai antar variabel tanpa memerlukan variabel sementara.

```
1  x, y = 1, 2 #penugasan simultan
2
3  var = 123
4  t1 = (1, )
5  t2 = (2, )
6  t3 = (3, var)
7  t1, t2, t3 = t2, t3, t1 #penugasan simultan
8  print(t1, t2, t3)
```

- Dari contoh di atas kita mendapat fakta baru bahwa **elemen tuple dapat berupa variabel.**

DICTIONARIES-1

- Dictionary merupakan struktur data pada Python. dictionary **bukan suatu tipe sequence** (tetapi dapat dengan mudah beradaptasi dengan proses sequence) dan bersifat **mutable**.

```
1 dictionary = {"cat": "kucing", "dog": "anjing", "horse": "kuda"}
2 nilai_alpro2 = {'morin': 90, 'arya': 95, 'faqih': 98}
3 dictionary_kosong = {}
4
5 print(dictionary)
6 print(nilai_alpro2)
7 print(dictionary_kosong)
```

- pada line 1, dictionary menggunakan kunci dan nilai yang keduanya berupa string (string-string). pada line 2, kuncinya berupa string sedangkan nilainya merupakan angka yaitu integer (string-angka).
- pasangan kunci-nilai juga memungkinkan untuk pasangan angka-string maupun angka-angka

DICTIONARIES-2

- Dictionary pada python bekerja dengan cara yang sama seperti kamus bilingual. dalam python kata yang dicari disebut kunci, dan kata yang didapatkan adalah nilai. sehingga dictionary merupakan kumpulan kunci-nilai.
- setiap kunci harus unik, tidak boleh ada lebih dari satu kunci dengan nilai sama
- kunci dapat berupa jenis objek yang tidak dapat diubah: bisa berupa angka (int/float), string, tetapi tidak bisa List.
- suatu dictionary bukan List. List merupakan sekumpulan nilai sedangkan dictionary menyimpan pasangan nilai.
- fungsi len() bisa digunakan juga untuk dictionary, untuk mengembalikan jumlah pasangan kunci-nilai dalam dictionary
- dictionary merupakan tools satu arah, artinya jika kalian memiliki kamus inggris-indonesia berarti kita hanya bisa mencari padanan kata bahasa inggris ke bahasa indonesia, tidak bisa sebaliknya.

DICTIONARIES-3

- hasil dari mencetak seluruh isi dari dictionary menggunakan fungsi print mungkin akan menghasilkan urutan pasangan kunci-nilai yang berbeda, tergantung versi python yang digunakan.
- karena dictionary bukanlah List, maka urutan di mana dictionary menyimpan data diluar kendali programmer. dan urutan ini tidak memiliki arti / pengaruh apapun (berbeda dengan list yang urutan itu penting untuk indexing karena List merupakan data sequence)
- untuk mengakses isi dictionary tertentu kita gunakan kunci, bukan index.

```
5 print(dictionary['cat'])  
6 print(nilai_alpro2['morin'])
```

- Jika key merupakan sebuah string, maka perlu diingat bahwa key merupakan case-sensitive. artinya string 'cat' berbeda dengan 'Cat'.

DICTIONARIES-4

- kita tidak bisa mencari menggunakan kunci yang tidak terdaftar pada dictionary

```
5 print(dictionary['lion'])
```

- ketika membuat dictionary dengan ekspresi yang panjang kita bisa menuliskannya secara vertical supaya lebih programmer-friendly. format seperti ini dinamakan **hanging indent**.

```
1 dictionary = {  
2     "cat": "kucing",  
3     "dog": "anjing",  
4     "horse": "kuda"  
5 }  
6 nilai_alpro2 = {'morin': 90,  
7     'arya': 95,  
8     'faqih': 98  
9 }
```


METHOD KEYS() DAN VALUES()

- karena dictionary bukan list maka untuk memindai isi dari dictionary tidak bisa menggunakan looping for seperti pada List atau Tuple.
- untuk memindai dictionary menggunakan looping for kita membutuhkan method dengan nama keys(). method ini mengembalikan objek yang dapat diiterasi yang terdiri dari semua kunci yang terkumpul dalam kamus. memiliki sekumpulan kunci dapat mempermudah ketika mengakses seluruh dictionary.

```
1 dictionary = {"cat": "kucing", "dog": "anjing", "horse": "kuda"}
2 for kunci in dictionary.keys():
3     print(kunci, "->", dictionary[kunci])
```

- method value bekerja sama seperti keys() tetapi mengembalikan nilai

```
for indo in dictionary.values():
    print(indo)
```


METHOD & FUNGSI LAINNYA-1

- Method items() mengembalikan tuple di mana setiap tuple merupakan pasangan kunci-nilai.

```
1 dictionary = {"cat": "kucing", "dog": "anjing", "horse": "kuda"}  
2 for eng, indo in dictionary.items():  
3     print(eng, "->", indo)
```

- method update() digunakan untuk menambahkan pasangan kunci-nilai baru pada dictionary

```
2 dictionary.update({'duck': 'bebek'})
```

- method popitem() digunakan untuk menghapus random item pada dictionary

```
3 dictionary.popitem()
```

METHOD & FUNGSI LAINNYA-2

tuple dapat diconvert menjadi dictionary menggunakan fungsi dict()

```
1 warna = (("hijau", "#008000"), ("biru", "#0000FF"))
2
3 kamus_warna = dict(warna)
4 print(kamus_warna)
```

menyalin dictionary: menggunakan method copy()

menghapus semua isi dictionary: menggunakan method clear()

MODIFIKASI DICTIONARY

memasukkan nilai baru ke kunci yang ada:

```
dictionary['cat'] = 'mpus'
```

mengurutkan isi dictionary:

```
for key in sorted(dictionary.keys()):  
    print(dictionary)
```

menambahkan pasangan kunci-nilai baru juga bisa dilakukan dengan cara lain:

```
10 dictionary['lion'] = 'singa'
```

jika method popitem() digunakan untuk menghapus item secara random, maka untuk menghapus item spesifik dilakukan dengan cara berikut:

```
11 del dictionary['dog']
```

CONTOH TUPLE-DICTIONARY

- jika kamu membutuhkan program untuk mengevaluasi nilai rerata mahasiswa
- program harus meminta nama dari mahasiswa dan nilainya
- memasukkan nama kosong berarti menyelesaikan proses input datanya. (ini akan menyebabkan error, tetapi nanti kita akan belajar exception)
- program akan menampilkan semua nama mahasiswa yang dimasukkan beserta nilai rerata dari setiap nama mahasiswa.
- kode beserta penjelasan dapat dilihat pada slide selanjutnya

CONTOH TUPLE-DICTIONARY

```
1 kelas_informatika = {} #membuat dict kosong
2 while True: #memasuki infinite looping
3     nama = input("Masukkan nama mahasiswa: ") #masukkan nama
4     if nama == '': #cek jika namanya kosong maka:
5         break #break
6     nilai = int(input("Masukkan nilai(0-10): ")) #masukan nilai
7     if nilai not in range(0, 11): #cek jika nilainya ga di antara 0 sampai <11 (0-10) maka:
8         break #break
9     if nama in kelas_informatika: #cek jika nama sudah ada di dict maka:
10        kelas_informatika[nama] += (nilai, ) #menambah nilai tuple baru
11    else: #jika nama tidak ada pada dict maka:
12        kelas_informatika[nama] = (nilai, ) #membuat tuple baru dengan nilai tunggal
13
14    for nama in sorted(kelas_informatika.keys()): #untuk setiap nama pada dict:
15        total_nilai = 0 #variabel untuk menampung total nilai setiap mhs
16        jml_nilai = 0 #variabel sebagai counter dari jml nilai setiap mhs
17        for nilai in kelas_informatika[nama]: #untuk setiap nilai di nama mhs di dict:
18            total_nilai += nilai #menjumlah kumulatif nilai yang dimasukkan
19            jml_nilai += 1 #menghitung jml nilai yang dimasukkan
20    print(nama, ":", total_nilai/jml_nilai) #menampilkan nama : rerata(total_nilai/jml_nilai)
```